

Proyecto I: Reconocimiento de comandos de voz y despliegue móvil con ONNX Runtime Mobile

Fabrizio Mena Mejía

Instituto Tecnológico de Costa Rica

Email: fabriciomena11@estudiantec.cr

Anthony Fuentes Calvo

Instituto Tecnológico de Costa Rica

Email: anthonyfuentescalvo@estudiantec.cr

Fernando Sánchez Hidalgo

Instituto Tecnológico de Costa Rica

Email: Fer.sanchez@estudiantec.cr

Abstract—

I. MARCO TEÓRICO

II. DISEÑO DE ARQUITECTURA DE IA

A. Control de Aleatoriedad y Reproducibilidad

Para garantizar la reproducibilidad de los experimentos, se implementó un estricto control de aleatoriedad. Se definió una semilla global (*seed* = 42) utilizada para inicializar los generadores de números pseudoaleatorios de las librerías *random*, *numpy* y *torch*. Adicionalmente, se configuró el comportamiento determinista de cuDNN desactivando el *benchmark* dinámico donde fuera necesario.

B. Modelo A: Datos Crudos (LeNet-5 Adaptado)

El Modelo A está basado en la arquitectura clásica de LeNet-5, pero fue adaptado específicamente para el procesamiento de mel-espectrogramas.

1) *Justificación de modificaciones:* Se realizaron los siguientes cambios sobre la arquitectura original de LeCun:

- **Tamaño de entrada:** Se modificó para recibir tensores de $1 \times 128 \times 128$ correspondientes a las imágenes generadas del audio, en lugar de los 32×32 originales.
- **Filtros convolucionales:** Se aumentó el número de canales a 32, 64 y 128 en las tres capas convolucionales consecutivas para capturar patrones espectrales más ricos y complejos.
- **Función de Activación:** Se preserva el uso de la función de activación clásica Tanh estipulada en la arquitectura original de LeNet-5. Esto se hace en estricto cumplimiento con los requerimientos del proyecto que limitan el uso de funciones más modernas como ReLU, garantizando así que el modelo mantenga la esencia matemática clásica de la red mientras se adapta al dominio de audio.
- **Padding:** En las capas convolucionales iniciales se utiliza un *padding*=0, consistentemente con la versión clásica de LeNet. Esta elección genera una reducción espacial después de cada convolución antes del AvgPool2d, lo que simplifica el diseño y mantiene la estructura de extracción de características sin introducir ceros adicionales en los bordes.
- **Clasificador 100% Convolucional:** Se reemplazó la sección tradicional de capas totalmente conectadas (*Fully Connected*) por una operación de *Global Average Pooling*

seguida de convoluciones 1×1 . Esta modificación reduce significativamente el número de parámetros y mitiga el sobreajuste (*overfitting*), preparando el modelo para su futuro despliegue en dispositivos móviles donde los parámetros en capas densas consumen demasiada memoria RAM.

2) *Definición de la Arquitectura:* La arquitectura final se resume en el diagrama presentado en la Tabla I. Consta de tres bloques convolucionales iniciales con *Average Pooling*, seguidos de una capa de *Global Average Pooling* y un bloque clasificador compuesto exclusivamente por convoluciones de 1×1 .

TABLE I
DIAGRAMA DE CAPAS - LEnET-5 ADAPTADO

Capa / Operación	Dimensión de Salida
Input (Mel-Spectrogram)	$(B, 1, 128, 128)$
Conv1 ($32 \times 5 \times 5$) + Tanh + AvgPool(2)	$(B, 32, 62, 62)$
Conv2 ($64 \times 5 \times 5$) + Tanh + AvgPool(2)	$(B, 64, 29, 29)$
Conv3 ($128 \times 5 \times 5$) + Tanh + AvgPool(2)	$(B, 128, 12, 12)$
Global Average Pooling	$(B, 128, 1, 1)$
Conv4 ($128 \rightarrow 512, 1 \times 1$) + Tanh	$(B, 512, 1, 1)$
Conv5 ($512 \rightarrow 128, 1 \times 1$) + Tanh	$(B, 128, 1, 1)$
Conv6 ($128 \rightarrow 10, 1 \times 1$) (Logits)	$(B, 10, 1, 1)$
Flatten Final	$(B, 10)$

3) *Función de Pérdida, Optimizador y Entrenamiento:* Se utilizó *Cross-Entropy Loss* como función de pérdida, la cual es ideal para clasificación multiclase. El optimizador seleccionado fue Adam con una tasa de aprendizaje de 1×10^{-3} y *weight decay* de 1×10^{-4} para mitigar el sobreajuste. Además, se empleó un *scheduler* (*ReduceLR0nPlateau*) que reduce la tasa de aprendizaje a la mitad si la precisión de validación no mejora en 3 épocas. Las rutinas de entrenamiento se ejecutaron usando precisión mixta nativa (*torch.amp*).

III. ANÁLISIS Y COMPARACIÓN DE RESULTADOS

A. Modelo A: Datos crudos

El Modelo A actual —con la arquitectura LeNet-5 adaptada a mel-espectrogramas, activaciones Tanh y un clasificador 100% convolucional— alcanzó los siguientes resultados en la evaluación final del conjunto de prueba:

- **Loss (test):** 1.0522
- **Accuracy (test):** 0.6388
- **F1 Macro (test):** 0.6344

images/Modelo_LeNet5_adaptado.png

Fig. 1. Diagrama visual de la arquitectura del Modelo A.

- **Mejor Accuracy de validación:** 0.6300

Estos valores reflejan el comportamiento del modelo actual con datos crudos, el desglose de resultados para cada comando de voz se muestra en la tabla II

TABLE II
REPORTE DE CLASIFICACIÓN - MODELO A (DATOS CRUDOS)

Comando	Precision	Recall	F1-Score	Support
yes	0.87	0.57	0.69	630
no	0.51	0.33	0.40	569
up	0.55	0.82	0.66	573
down	0.54	0.77	0.64	586
left	0.61	0.70	0.65	555
right	0.83	0.72	0.77	576
on	0.68	0.67	0.67	574
off	0.69	0.49	0.57	565
stop	0.65	0.73	0.69	558
go	0.59	0.60	0.59	597
Accuracy		0.64		5783
Macro Avg	0.65	0.64	0.63	5783

En la gráfica de **Loss**, la curva de entrenamiento desciende de forma consistente desde aproximadamente 2.19 hasta 1.22, mientras que la pérdida de validación se reduce desde 2.05 hasta 1.24. Este comportamiento indica que el modelo aprende patrones útiles sin incurrir en un sobreajuste abrupto durante la fase final del entrenamiento.

La gráfica de **Accuracy** muestra una tendencia de aprendizaje gradual: la precisión de entrenamiento crece de alrededor de 16% a 58%, mientras que la precisión de validación alcanza un pico de 63% en las primeras épocas y luego se estabiliza cerca de 59%. Esto sugiere que el modelo captura información discriminativa con el conjunto actual, pero

images/Grafica_Loss_Modelo_A_raw.png

Fig. 2. Pérdida de entrenamiento y validación para el Modelo A.

images/Grafica_Accuracy_Modelo_A_raw.png

Fig. 3. Precisión de entrenamiento y validación para el Modelo A.

su capacidad de generalización permanece limitada por la complejidad del problema y por el uso de datos crudos sin augmentation.



Fig. 4. Evolución del F1-Score macro en validación para el Modelo A.

La gráfica de **F1-Score macro** evidencia que el desempeño del modelo mejora rápidamente en las primeras épocas y luego se estabiliza, reflejando un balance moderado entre precisión y recall. El valor final de F1 macro en prueba es 0.6344, lo que confirma que aún existen diferencias sustanciales entre clases y que algunos comandos son más difíciles de distinguir.

La matriz de confusión muestra que los comandos *yes* y *no* presentan confusiones relevantes, al igual que pares como *up/down* y *go/stop*. Esto es consistente con el desempeño de la arquitectura actual: las activaciones Tanh y el clasificador 100% convolucional mantienen un modelo compacto, pero la representación aprendida todavía requiere mayor robustez frente a variaciones acústicas.

El rediseño del clasificador con convoluciones 1×1 y Global Average Pooling reduce parámetros y favorece el despliegue móvil, aunque la versión actual del modelo demuestra que se necesita complementar esta eficiencia con técnicas adicionales como *data augmentation*, normalización avanzada o ajustes finos de hiperparámetros para aumentar la precisión hacia el 70% o superior.

IV. REFERENCIAS

REFERENCES

- [1] TensorFlow, “TensorFlow Lite,” 2026. [Online]. Available: <https://www.tensorflow.org/lite/guide>. [Accessed: May 2, 2026].
- [2] TensorFlow Datasets, “speech_commands,” 2026. [Online]. Available: https://www.tensorflow.org/datasets/catalog/speech_commands. [Accessed: May 2, 2026].
- [3] PyTorch, “torchaudio,” 2026. [Online]. Available: <https://docs.pytorch.org/audio/stable/index.html>. [Accessed: May 2, 2026].



Fig. 5. Matriz de confusión del Modelo A (datos crudos) sobre los datos de prueba.

- [4] Android Developers, “MediaRecorder,” 2026. [Online]. Available: <https://developer.android.com/reference/android/media/MediaRecorder>. [Accessed: May 2, 2026].
- [5] Android Developers, “Get started with Jetpack Compose,” 2026. [Online]. Available: <https://developer.android.com/develop/ui/compose/documentation>. [Accessed: May 2, 2026].